

LAPORAN TUGAS MAGANG APTRG

Misi GUIDED menggunakan SITL dan Pymavlink

- **Nama:** Achmad Raffa Adhyaksa
- **NIM:** 103012400200
- **S1 Informatika**

1. Persiapan Jaringan (WSL2 ke Windows)

Kendala utama pada simulasi ini adalah merutekan paket UDP dari environment Ubuntu (WSL2) ke Mission Planner di Windows. Langkah penyelesaian:

- Buka terminal Ubuntu (WSL2).
- Cari IP Gateway (IP Windows) dengan perintah:

```
(venv-ardupilot) raffa_f8rbz1m@SuperMomoo:~$ ip route  
| grep default  
default via 172.27.144.1 dev eth0 proto kernel  
(venv-ardupilot) raffa_f8rbz1m@SuperMomoo:~$ |
```

`ip route | grep default`

IP yang didapat (**172.27.144.1**). Ini akan digunakan sebagai jalur komunikasi dari SITL ke Mission Planner.

2. Menjalankan SITL (Software In The Loop)

Jalankan simulator ArduCopter di terminal Ubuntu. Lakukan *split* jalur UDP agar tidak terjadi *port clashing* antara Mission Planner dan script Python.

- Jalankan perintah berikut (ganti **<IP_WINDOWS>** dengan IP dari langkah 1):

```
sim_vehicle.py -v ArduCopter -f quad --out=udp:<IP_WINDOWS>:14550  
--out=udp:127.0.0.1:14551
```

Tunggu hingga sistem *booting* selesai dan EKF3 mendeteksi GPS.

```
raffa_f8rbz1m@SuperMormoo  x  Ubuntu
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
*Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
*
AP: ArduPilot Ready
AP: AHRS: DCM active
AP: RC7: SaveWaypoint LOW
AP: EKF3 IMU0 initialised
AP: EKF3 IMU1 initialised
AP: AHRS: EKF3 active
AP: EKF3 IMU1 tilt alignment complete
AP: EKF3 IMU0 tilt alignment complete
AP: EKF3 IMU1 MAG0 initial yaw alignment complete
AP: EKF3 IMU0 MAG0 initial yaw alignment complete
AP: GPS 1: probing for u-blox at 230400 baud
AP: GPS 1: detected u-blox
fence present
Flight battery 100 percent
AP: EKF3 IMU1 origin set
AP: Field Elevation Set: 584m
AP: EKF3 IMU0 origin set
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: SET_MESSAGE_INTERVAL: ACCEPTED
Got COMMAND_ACK: SET_MESSAGE_INTERVAL: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
AP: EKF3 IMU0 is using GPS
AP: EKF3 IMU1 is using GPS
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: SET_MESSAGE_INTERVAL: ACCEPTED
Got COMMAND_ACK: SET_MESSAGE_INTERVAL: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Flight battery 100 percent
Got COMMAND_ACK: SET_MESSAGE_INTERVAL: ACCEPTED
Got COMMAND_ACK: SET_MESSAGE_INTERVAL: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
Got COMMAND_ACK: REQUEST_MESSAGE: ACCEPTED
MAV> Flight battery 100 percent
```

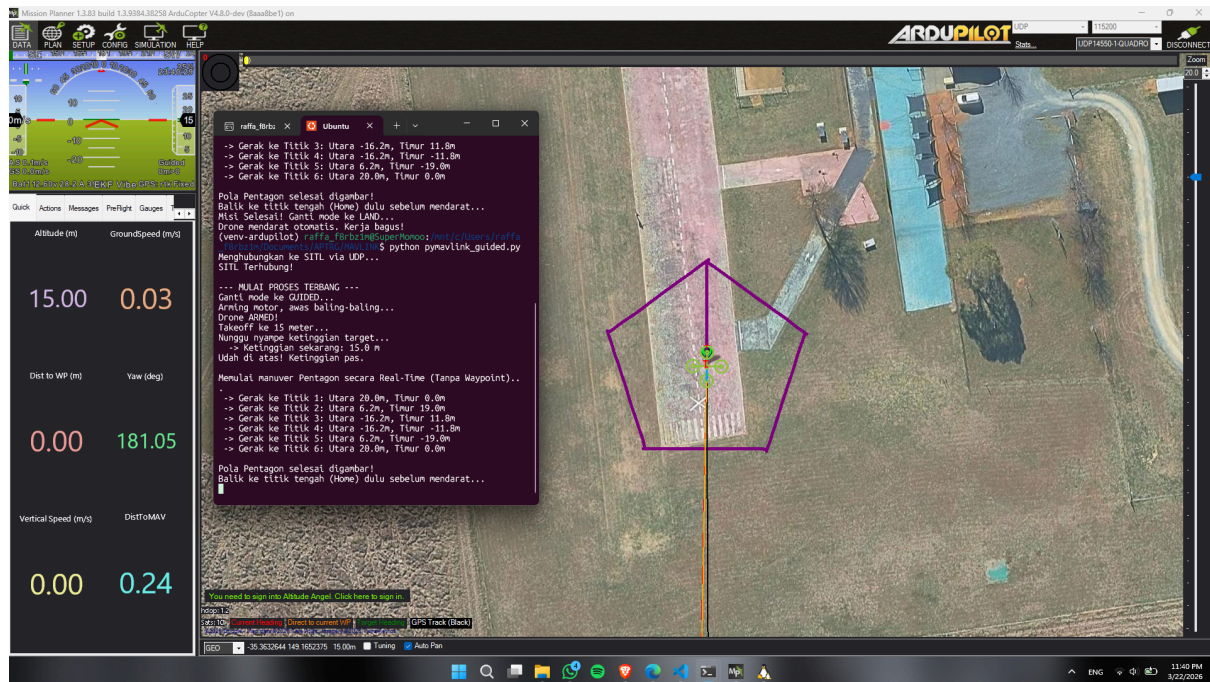
3. Koneksi Mission Planner

- Buka Mission Planner di OS Windows.
- Pada pojok kanan atas, pilih opsi **UDP**.
- Klik **Connect**, lalu masukkan port **14550**.
- Tunggu hingga parameter selesai dimuat dan posisi drone virtual muncul di peta.

4. Pembuatan Script Navigasi (Pymavlink)

Buat script Python untuk mengirimkan perintah target posisi lokal (sistem NED) secara *real-time* untuk membentuk pola Pentagon, melakukan *arming*, *takeoff*, dan mengeksekusi

seluruh manuver pergerakan dengan tetap berada di mode GUIDED."



```

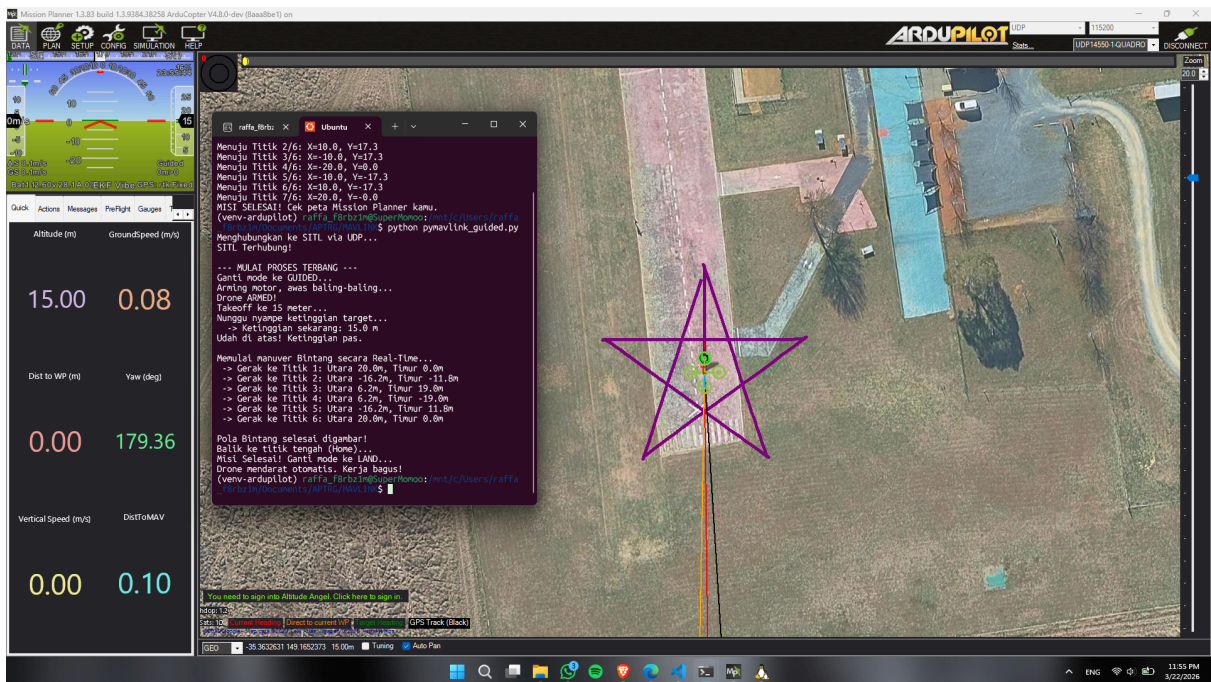
Welcome X fly_shape.py drone_shape.py pymavlink_segitiga.py pymavlink_guided.py X
pymavlink_guided.py > goto_position_target_local_ned
1 from pymavlink import mavutil
2 import time
3 import math
4
5 # =====
6 # 1. KONEKSI UDP KE SITL
7 # =====
8 print("Menghubungkan ke SITL via UDP...")
9 master = mavutil.mavlink_connection('udp:127.0.0.1:14551')
10 master.wait_heartbeat()
11 print("SITL Terhubung!")
12
13 # Fungsi untuk mengirim target posisi lokal (NED)
14 # Trae: Explain | Doc | X
15 def goto_position_target_local_ned(north, east, down):
16     msg = master.mav.set_position_target_local_ned_encode(
17         0, master.target_system, master.target_component,
18         mavutil.mavlink.MAV_FRAME_LOCAL_NED,
19         0b0000111111111000,
20         north, east, down,
21         0, 0, 0, 0, 0, 0, 0, 0)
22     master.mav.send(msg)
23
24 # =====
25 # 2. EKSEKUSI TERBANG (TAKEOFF)
26 # =====
27 print("\n--- MULAI PROSES TERBANG ---")
28 print("Ganti mode ke GUIDED...")
29 master.mav.set_mode_send(master.target_system, mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED, master.mode_mapping()[ 'GUIDED' ])
30 time.sleep(1)
31
32 print("Arming motor, awas baling-baling...")
33 master.mav.command_long_send(master.target_system, master.target_component,
34                               mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM, 0, 1, 0, 0, 0, 0, 0, 0)
35 master.motors_armed_wait()
36 print("Drone ARMED!")
37
38 target_alt = 15
39 print(f"Takeoff ke {target_alt} meter...")
40 master.mav.command_long_send(master.target_system, master.target_component,
41                               mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, 0, 0, 0, 0, 0, 0, 0, target_alt)
42
43 print("Nunggu nyampe ketinggian target...")
44 while True:
45     msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
46     if msg:
47         alt = msg.relative_alt / 1000.0
48         print(f" -> Ketinggian sekarang: {alt:.1f} m", end='\n')

```

```

Welcome fly_shape.py drone_shape.py pymavlink_segitiga.py pymavlink_guided.py X
pymavlink_guided.py > goto_position_target_local_ned
master.mav.set_mode_send(master.target_system, master.target_component,
mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, 0, 0, 0, 0, 0, 0, 0, target_alt)
40
41
42 print("Nunggu nyampe ketinggian target...")
43 while True:
44     msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
45     if msg:
46         alt = msg.relative_alt / 1000.0
47         print(f" -> Ketinggian sekarang: {alt:.1f} m", end='\n')
48         if alt >= (target_alt * 0.95):
49             print("\nUdah di atas! Ketinggian pas.")
50             break
51     time.sleep(0.1)
52
53 # =====
54 # 3. MANUEVER BINTANG LEBIH EFISIEN (PRE-KALKULASI)
55 # =====
56 radius_m = 20
57 waktu_tunggu = 10
58 print("\nMemulai manuver Bintang secara Real-Time...")
59
60 # Menghitung semua titik di awal (List Comprehension)
61 # Lompat 144 derajat tiap titik. range(6) buat 5 titik + 1 balik ke awal biar tertutup rapat.
62 # Ditambah offset 90 derajat biar ujung lancip bintang menghadap persis ke Utara.
63 titik_bintang = [
64     (radius_m * math.sin(math.radians(90 + ((i % 5) * 144))),
65      radius_m * math.cos(math.radians(90 + ((i % 5) * 144))))
66     for i in range(6)
67 ]
68
69 # Eksekusi pergerakan
70 for i, (north, east) in enumerate(titik_bintang):
71     print(f" -> Gerak ke Titik {i+1}: Utara {north:.1f}m, Timur {east:.1f}m")
72     goto_position_target_local_ned(north, east, -target_alt)
73     time.sleep(waktu_tunggu)
74
75 print("\nPola Bintang selesai digambar!")
76
77 # =====
78 # 4. LANDING
79 # =====
80 print("Balik ke titik tengah (Home)...")
81 goto_position_target_local_ned(0, 0, -target_alt)
82 time.sleep(8)
83
84 print("Misi Selesai! Ganti mode ke LAND...")
85 master.mav.set_mode_send(master.target_system, master.target_component, mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE
86 print("Drone mendarat otomatis. Kerja bagus!")

```



KODE KALO WAYPOINT

```
Welcome  fly_shape.py  drone_shape.py X  pymavlink_segitiga.py X
pymavlink_segitiga.py > ...
1  from pymavlink import mavutil
2  import time
3  import math # Diperlukan untuk perhitungan trigonometri (sin/cos) pada pola sudut
4
5  # =====
6  # 1. INISIALISASI KONEKSI UDP KE SITL
7  # =====
8  print("Menghubungkan ke SITL via UDP...")
9  # Menggunakan localhost dan port 14551 sesuai pembagian routing di WSL
10 master = mavutil.mavlink_connection('udp:127.0.0.1:14551')
11 master.wait_heartbeat()
12 print("SITL Terhubung!")
13
14 # =====
15 # 2. MENDAPATKAN KOORDINAT HOME (TITIK ACUAN)
16 # =====
17 print("Menunggu GPS lock (Bisa agak lama, sabar ya)...")
18 while True:
19     # Membaca pesan GLOBAL_POSITION_INT untuk mendapatkan koordinat awal
20     msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
21     if msg:
22         home_lat = msg.lat # Koordinat latitude dalam format 1e7
23         home_lon = msg.lon # Koordinat longitude dalam format 1e7
24         break
25 print(f"Home terkunci di Lat: {home_lat}, Lon: {home_lon}")
26
27 # =====
28 # 3. KALKULASI TITIK WAYPOINT (POLA PENTAGON)
29 # =====
30 # Konversi meter ke format MAVLink 1e7 (1 meter ~ 89.8 satuan di 1e7)
31 meter_to_1e7 = 1e7 / 111320.0
32 radius_m = 20 # Radius jarak pentagon sejauh 20 meter dari titik tengah
33 target_alt = 15 # Ketinggian terbang 15 meter
34
35 waypoints = [
36     # Sequence 0: Titik Dummy Home (Wajib ada pada protokol ArduPilot)
37     {'cmd': mavutil.mavlink.MAV_CMD_NAV_WAYPOINT, 'x': home_lat, 'y': home_lon, 'z': 0},
38     # Sequence 1: Perintah Takeoff tegak lurus ke ketinggian target
39     {'cmd': mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, 'x': home_lat, 'y': home_lon, 'z': target_alt},
40 ]
41
42 # Looping untuk membuat 5 titik sudut Segi Lima (Pentagon)
43 for i in range(5):
44     # Sudut segi lima adalah 360 / 5 = 72 derajat per sudut
45     angle_deg = i * 72
46     angle_rad = math.radians(angle_deg) # Konversi derajat ke radian untuk fungsi trigonometri
47
48     # Menghitung offset X (Latitude) dan Y (Longitude) menggunakan Sin dan Cos
```

```

Welcome  fly_shape.py  drone_shape.py  pymavlink_segitiga.py X
pymavlink_segitiga.py > ...
48     # Menghitung offset X (Latitude) dan Y (Longitude) menggunakan Sin dan Cos
49     dx = radius_m * math.cos(angle_rad)
50     dy = radius_m * math.sin(angle_rad)
51
52     # Menerapkan offset ke koordinat home awal
53     wp_lat = int(home_lat + (dx * meter_to_1e7))
54     wp_lon = int(home_lon + (dy * meter_to_1e7))
55
56     # Memasukkan titik yang sudah dihitung ke dalam list waypoint
57     waypoints.append({'cmd': mavutil.mavlink.MAV_CMD_NAV_WAYPOINT, 'x': wp_lat, 'y': wp_lon, 'z': target_alt})
58
59 # Sequence 7: Mengunci pola dengan mengulangi titik pertama pentagon (agar garis menyambung tertutup)
60 waypoints.append(waypoints[2])
61
62 # Sequence 8: Perintah RTL (Return To Launch) untuk pendaratan otomatis setelah misi selesai
63 waypoints.append({'cmd': mavutil.mavlink.MAV_CMD_NAV_RETURN_TO_LAUNCH, 'x': 0, 'y': 0, 'z': 0})
64
65 # =====
66 # 4. PROTOKOL UPLOAD MISI (MAVLINK HANDSHAKE)
67 # =====
68 print(f"Memulai upload {len(waypoints)} titik misi ke SITL...")
69 # Mengirim jumlah total waypoint ke sistem drone
70 master.mav.mission_count_send(master.target_system, master.target_component, len(waypoints))
71
72 for i in range(len(waypoints)):
73     # Menunggu request dari drone untuk setiap urutan (sequence) waypoint
74     msg = master.recv_match(type=['MISSION_REQUEST_INT', 'MISSION_REQUEST'], blocking=True, timeout=5)
75     if not msg:
76         print("Gagal dapat request dari SITL! Coba jalanin ulang scriptnya.")
77         exit()
78
79     seq = msg.seq
80     wp = waypoints[seq]
81
82     # Mengirim detail koordinat untuk sequence yang direquest
83     master.mav.mission_item_int_send(
84         master.target_system, master.target_component,
85         seq,
86         mavutil.mavlink.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT, # Menggunakan frame altitude relatif terhadap daratan
87         wp['cmd'],
88         0, 1, # Current (0) & Autocontinue (1)
89         0, 0, 0, 0, # Parameter 1-4 tidak digunakan untuk waypoint standar
90         wp['x'], wp['y'], wp['z']
91     )
92     print(f" -> Titik {seq} berhasil dikirim!")
93
94 # Menunggu konfirmasi bahwa seluruh rute berhasil diterima oleh sistem (ACK 0)
95 ack = master.recv_match(type='MISSION_ACK', blocking=True, timeout=5)
96 print(f"Upload selesai! Status ACK: {ack.type} if ack else 'No ACK!')

```

```

Welcome  fly_shape.py  drone_shape.py  pymavlink_segitiga.py X
pymavlink_segitiga.py > ...
96  print(f"Upload selesai! Status ACK: {ack.type if ack else 'No ACK'}")
97
98  # =====
99  # 5. EKSEKUSI PENERBANGAN OTOMATIS
100 # =====
101 print("\n--- MULAI PROSES TERBANG ---")
102
103 # Mengubah mode ke GUIDED sebagai syarat sebelum Arming di ArduPilot
104 print("Ganti mode ke GUIDED...")
105 mode_id_guided = master.mode_mapping()['GUIDED']
106 master.mav.set_mode_send(master.target_system, mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED, mode_id_guided)
107 time.sleep(1)
108
109 # Mengirim perintah ARM untuk menyalakan motor baling-baling
110 print("Arming motor, awas baling-baling...")
111 master.mav.command_long_send(master.target_system, master.target_component,
112                               mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM, 0, 1, 0, 0, 0, 0, 0, 0)
113
114 # Memastikan motor benar-benar menyala sebelum melanjutkan
115 master.motors_armed_wait()
116 print("Drone ARMED! Siap ngacir...")
117
118 # Mengirim perintah Takeoff sesuai target altitude
119 print(f"Takeoff ke {target_alt} meter...")
120 master.mav.command_long_send(master.target_system, master.target_component,
121                               mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, 0, 0, 0, 0, 0, 0, 0, target_alt)
122
123 # Loop untuk memantau ketinggian secara real-time
124 print("Nunggu nyampe ketinggian target...")
125 while True:
126     msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
127     if msg:
128         alt = msg.relative_alt / 1000.0 # Konversi milimeter ke meter
129         print(f" -> Ketinggian sekarang: {alt:.1f} m", end='\r')
130         # Jika ketinggian sudah mencapai 95% dari target, proses dilanjutkan
131         if alt >= (target_alt * 0.95):
132             print("\nUdah di atas! Ketinggian pas.")
133             break
134         time.sleep(0.1)
135
136 # Mengubah mode ke AUTO agar drone bergerak menyusuri titik-titik misi
137 print("Ganti mode ke AUTO biar ngikutin pola PENTAGON...")
138 master.set_mode('AUTO')
139
140 print("\n=== DRONE SEDANG MENJALANKAN MISI PENTAGON! ===")
141 print("Cek Mission Planner, terus klik 'Read' di tab PLAN untuk memperbarui visualisasi garis.")
142 print("Tekan Ctrl+C di terminal ini untuk menghentikan program.")
143

```

```

143
144 # Loop agar program tetap berjalan dan drone menyelesaikan misinya
145 try:
146     while True:
147         time.sleep(1)
148 except KeyboardInterrupt:
149     print("\nProgram dihentikan secara manual.")

```

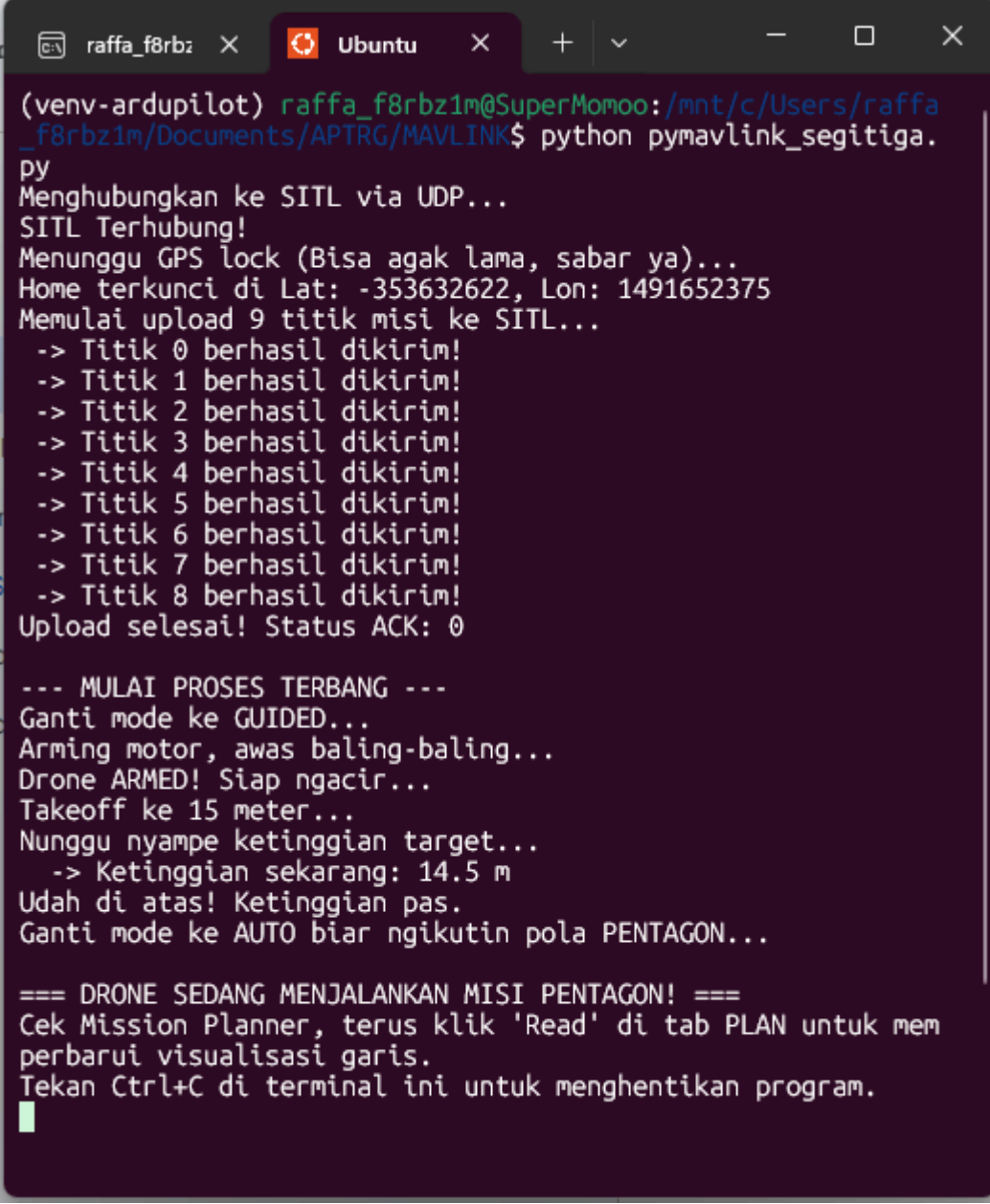
- Pastikan koneksi di dalam script mengarah ke port lokal Ubuntu:

master = mavutil.mavlink_connection('udp:127.0.0.1:14551')

5. Eksekusi Misi

- Buka terminal Ubuntu baru.
- Jalankan script Python:

`python pymavlink_segitiga.py`



```
(venv-ardupilot) raffa_f8rbz1m@SuperMomoo:/mnt/c/Users/raffa_f8rbz1m/Documents/APTRG/MAVLINK$ python pymavlink_segitiga.py
Menghubungkan ke SITL via UDP...
SITL Terhubung!
Menunggu GPS lock (Bisa agak lama, sabar ya)...
Home terkunci di Lat: -353632622, Lon: 1491652375
Memulai upload 9 titik misi ke SITL...
-> Titik 0 berhasil dikirim!
-> Titik 1 berhasil dikirim!
-> Titik 2 berhasil dikirim!
-> Titik 3 berhasil dikirim!
-> Titik 4 berhasil dikirim!
-> Titik 5 berhasil dikirim!
-> Titik 6 berhasil dikirim!
-> Titik 7 berhasil dikirim!
-> Titik 8 berhasil dikirim!
Upload selesai! Status ACK: 0

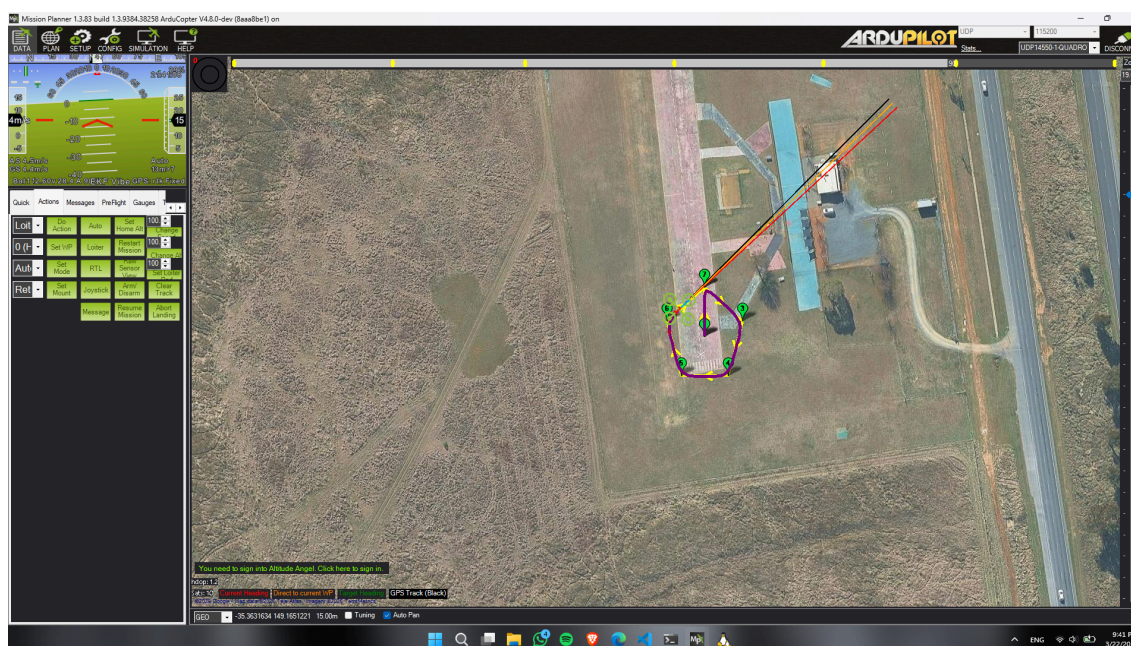
--- MULAI PROSES TERBANG ---
Ganti mode ke GUIDED...
Arming motor, awas baling-baling...
Drone ARMED! Siap ngacir...
Takeoff ke 15 meter...
Nunggu nyampe ketinggian target...
-> Ketinggian sekarang: 14.5 m
Udah di atas! Ketinggian pas.
Ganti mode ke AUTO biar ngikutin pola PENTAGON...

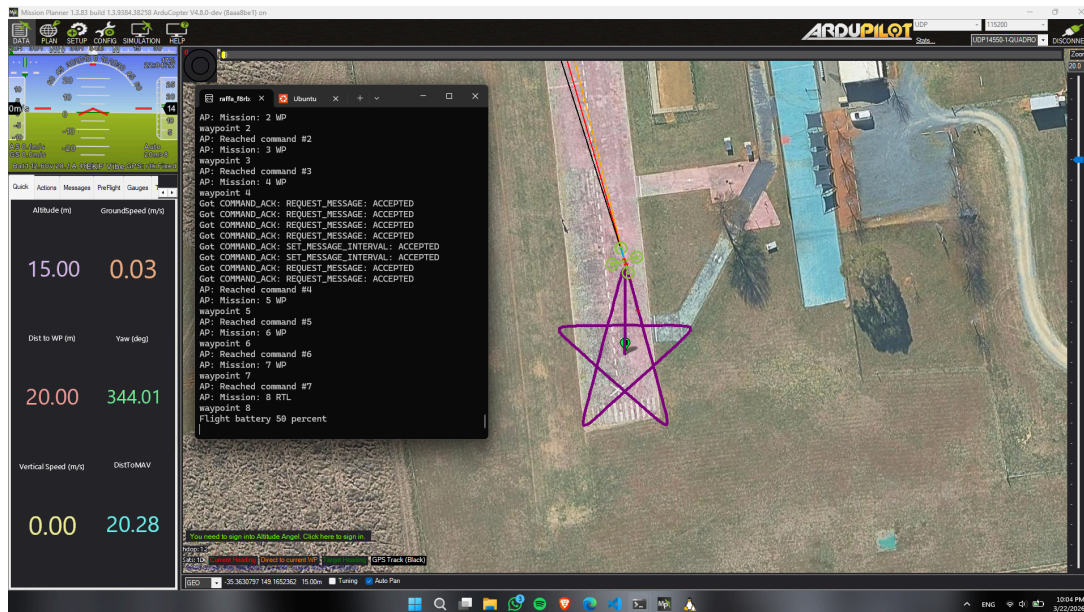
=== DRONE SEDANG MENJALANKAN MISI PENTAGON! ===
Cek Mission Planner, terus klik 'Read' di tab PLAN untuk mem
perbarui visualisasi garis.
Tekan Ctrl+C di terminal ini untuk menghentikan program.
```

Perhatikan output di terminal hingga muncul tulisan **"Upload selesai! Status ACK: 0"**.

Segera buka Mission Planner, masuk ke tab **PLAN**, lalu klik tombol **Read** di sebelah kanan agar garis pola misi Pentagon tergambar di peta.

Drone akan otomatis *takeoff* ke ketinggian target (15 meter), menelusuri 5 titik sudut pentagon, dan kembali ke titik awal (RTL).





Selain mengontrol drone secara *real-time* menggunakan mode GUIDED, eksplorasi tambahan dilakukan dengan merancang rute terprogram menggunakan sistem **Waypoint (Mode AUTO)**.

Perbedaan Utama:

1. **Mode GUIDED:** Pergerakan diinstruksikan secara langsung (*real-time*) menggunakan koordinat lokal NED (*North, East, Down*) melalui perintah `SET_POSITION_TARGET_LOCAL_NED`. Mission Planner hanya menampilkan pergerakan aktual ikon drone tanpa garis rute.
2. **Mode AUTO (Waypoint):** Rute titik koordinat dihitung dan diunggah (*upload*) ke memori drone sebelum terbang menggunakan perintah `MAV_CMD_NAV_WAYPOINT`.

Pada mode AUTO, karena seluruh titik misi sudah tersimpan di memori sistem, Mission Planner **dapat menampilkan garis rute lintasan di peta**. Pendekatan ini sangat cocok untuk misi penerbangan otonom yang sudah terencana secara statis (*pre-planned*) dengan presisi bentuk yang tinggi, seperti pemetaan wilayah atau patroli.